

AgentWorld Async — 通俗架构说明

1. 这是什么？

一个用 Python 写的虚拟世界。里面的人（Agent）会自己决定做什么——渴了去喝酒、累了坐下、看见朋友过去聊天。不用人操控。背后用 AI（DeepSeek / MiniMax）当大脑。每个 Agent 内部有一个 AI 帮他判断："我现在很渴，旁边有个酒吧，酒 5 块钱一杯，我有 30 块钱，要不要买一杯？"

2. 三个比喻理解核心

比喻 1：Layer 像功能外套

每个人/物都有几件"功能外套"。吧台穿了两件：一件让别人能看到它（visual），一件让别人能跟它交互（interaction）。酒也穿了两件，但"看到它"那件外套比较小——要离得很近才能看见那杯酒。

Agent（杰洛特）多穿了一件"自主外套"（agent）——他会自己走路、自己决策。

比喻 2：Entity 像通用容器

世界里的所有东西——吧台、麦酒、杰洛特、叶奈法、公告板——全部用同一个盒子来装。区别只是盒子里放了哪些功能外套。这意味着你要加一颗"苹果"进去，不需要写苹果类。只需要在 YAML 文件里写：苹果在什么位置、有什么外套、能做什么。零代码。

比喻 3：Systems 像调度中心

外套之间不互相联系。visual 外套不知道 interaction 外套的存在。所有的"跨外套"操作——比如"我看到一个东西"需要同时查 visual 外套和我的自主外套——交给 Systems 来处理。

这就像机场调度塔：飞行员不直接跟地勤对话，全通过塔台。

3. 杰洛特的一天（一步一步）

初始状态：杰洛特在广场 (7,5)，thirst=71（渴了），coins=150（有钱）。

* 第 1 步 — 感知 (SENSE)

SensorySystem 扫描周围所有实体：

* 水井 (7,6) — 可以打水

o 酒馆入口 (13,5) — 距离 6，看得见够不着

o 商人摊位 (10,3) — 距离 5

* 第 2 步 — 决策 (LLM #1)

杰洛特的 AI 大脑收到这些信息 + 自己的状态：

"我渴了 (thirst=71)，旁边有水井。先去打水喝，再去酒馆喝一杯解乏。"

输出: { move_to: [7,6], target_entity: "square_well", action: "打水" }

* 第 3 步 — 执行

-> 移动: [7,5] -> [7,6]

-> 交互: 打水 (resolve=rule -> 引擎直接算 -> thirst -30)

* 第 4 步 — 再决策

thirst=41（不渴了）。fun=22（无聊）。social=25（孤单）。

"去酒馆喝一杯，顺便看看公告板。"

输出: { move_to: [13,5] }

* 第 5 步 — 进酒馆

走到酒馆入口 -> 交互: "进入酒馆"

-> gate 传送: zone=bar_zone, pos=(1,4)
-> sensory 清空重建 -> 看到吧台、麦酒、丹德里恩

* 第 6 步 — 喝麦酒
在吧台前 -> 交互: 麦酒.饮用
-> LLM #2 裁判介入:
看到: 杰洛特(thirst=41,coins=150) + 麦酒(price=5)
+ 吧台(老板在,mood=60)
-> 裁定: {thirst:-20, coins:-5, mood:+5}
-> 同时: 吧台老板 mood+2 (客人付钱了)

4. 喝麦酒到底发生了什么？

分两条线同时进行：

前台（杰洛特自己的循环）

- ① 决策: "喝麦酒" -> 提交交互 -> 标记自己为 "busy" (不能同时做别的事)
- ② busy 期间: 继续呼吸 (decay)、继续看周围 (sense)、收发 inbox 消息。但不能再移动或新交互。
- ③ 收到裁判结果: 应用属性变化 (thirst-20, coins-5) -> 记录记忆 -> 标记 idle

后台（InteractionSystem）

- ① 收集周边实体: "麦酒旁边是吧台，吧台后面是矮人老板，老板 mood=60"
- ② 组裁判 Prompt: 杰洛特的私有状态 + 麦酒的私有状态 + 周边实体信息
- ③ AI 裁判裁定: "杰洛特付出 5 块钱，喝下麦酒。老板看到客人付钱，心情好了一点。"
- ④ 结果投递到杰洛特的 busy_result -> 前台循环处理

5. 外部 Agent 怎么接入？（3 步）



6. 前端看什么

浏览器打开可以看到一个像素风格的地图。每个小人是彩色方块，走动时会有平滑动画。小人头上会弹出气泡文字。右边有事件日志滚动。

前端对世界内容一无所知。它只是按后端推送的 JSON 来摆 tile 和画精灵。加了新 zone、新实体、新精灵——前端一行代码不用改。

7. 和旧版 AgentWorld (v1) 的区别

	v1	v2 (现在)
时间	回合制: 所有人同时行动	异步: 各走各的节奏
Agent 怎么动	LLM 输出图操作指令	LLM 叙事决策 + 引擎执行
交互谁决定	预定义的cost/effects	LLM 裁判见双方私有状态裁定
实体定义	多个 Python 子类	唯一 Entity + Layer, 纯 YAML
关系	图边 (npc->item qty=5)	位置 (同坐标 = 有关联, 无父子)
前端	脚本输出 -> REPORT.md	Phaser.js 像素风实时渲染

8. 怎么加新东西？

三个例子展示"零代码新增"：

例子 1: 加一碟花生

```
# config/world.yaml
- id: peanuts
  name: "花生"
  zone: bar_zone
  pos: [7, 3]      # 吧台旁边
  visual:
    visible_radius: 3
    sprite: null
    info: {look: "一碟盐烤花生"}
  interaction:
    interaction_radius: 2
    private_attrs: {price: 2}
  actions:
    吃:
      target_type: passive
      resolve: rule
      rule:
        effects: {hunger: -15}
        narrative: "{caller}拈了几颗花生丢进嘴里，咸香脆口"
```

-> Python 代码零改动。LLM 自动在 prompt 中看到花生，Agent 饿了自己会去吃。

例子 2: 加一个新 NPC (希里)

```
- id: ciri
  name: "希里"
  zone: square
  pos: [9, 6]
  visual: {...}
  interaction:
    actions:
      交谈: {target_type: agent}
      切磋: {target_type: passive, resolve: llm}
  agent:
    autonomous: true
    personality: "辛特拉幼狮，正在躲避狂猎的追捕..."
```

```
drives:
  hunger: {decay: 0.02}
  thirst: {decay: 0.025}
```

例子 3: 加一个新 zone (凯尔莫罕)

```
zones:
- id: kaer_morhen
  name: "凯尔莫罕"
  width: 20
  height: 14
# + connections + gate entities + 实体

-> Python 代码零改动。前端加载时自动渲染。
```

9. Godot 前端计划

当前前端用 Phaser.js (网页)。计划升级到 Godot 4.x 游戏引擎。后端一行不改。

升级后的提升：

- 真正的像素精灵动画（走路 4 方向 × 4 帧）

- 碰撞

改动范围：只替换 web/ 目录。src/ 和 config/ 全部不变。

10. 技术要点速查

指标	数值
源文件	27 个 .py + 3 个 .yaml + 6 个 web/
LLM 调用点	2 个 (决策 + 裁判)
Agent 循环	while alive: decay -> sense -> think -> move -> interact ->
交互模式	3 种 (rule引擎 / llm裁判 / inbox消息)
API 端点	8 个 REST + 1 个 WebSocket
测试规模	3 zones × 23 entities × 3 NPCs, 21/21 属性合规
扩展方式	只改 YAML, 不改 Python